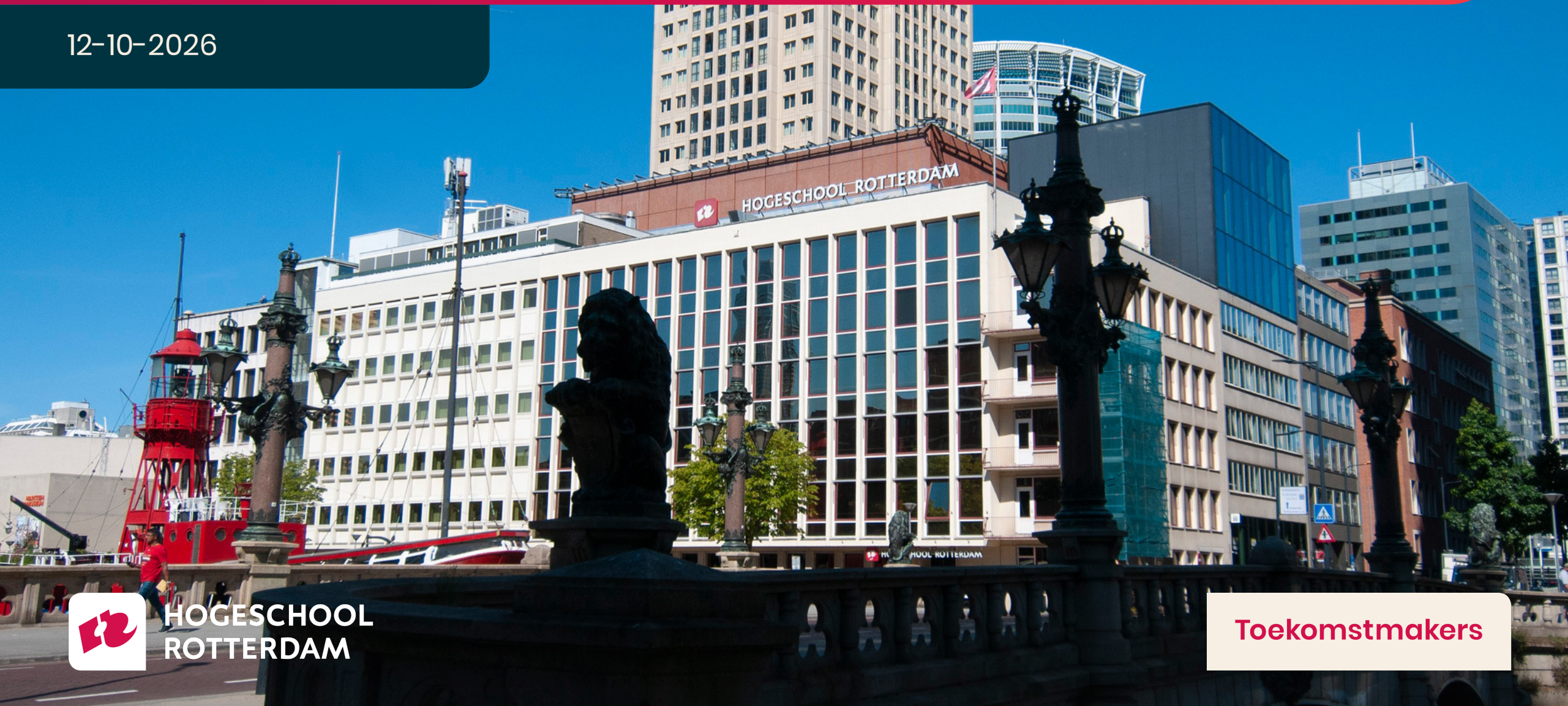
Technische Informatica: High-performance Computing

Les 16: Introductie shaders

12-10-2026

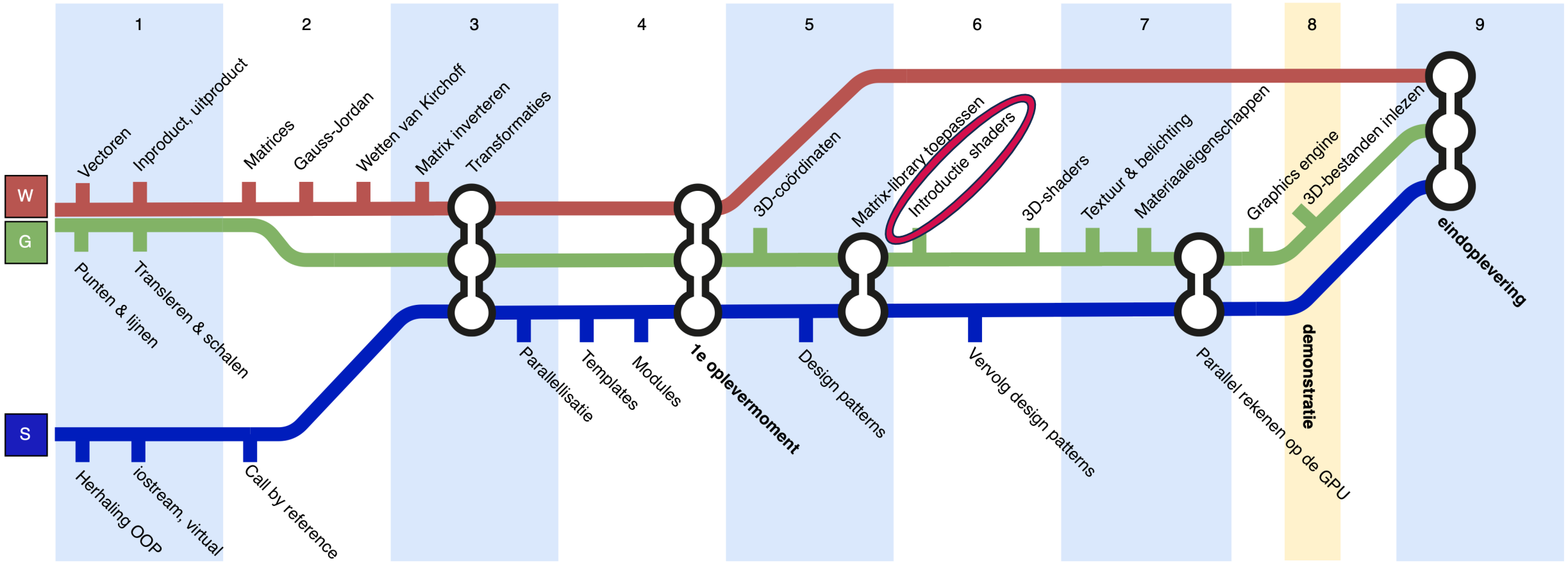


HOGESCHOOL ROTTERDAM

Toekomstmakers

Quiz

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
3D-coördinaten
Matrix-library toepassen
Introductie shaders
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Graphics engine
3D-bestanden inlezen
eindoplevering
Punten & lijnen
Transleren & schalen
Herhaling OOP
iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
Design patterns
Vervolg design patterns
Parallel rekenen op de GPU
demonstratie

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.



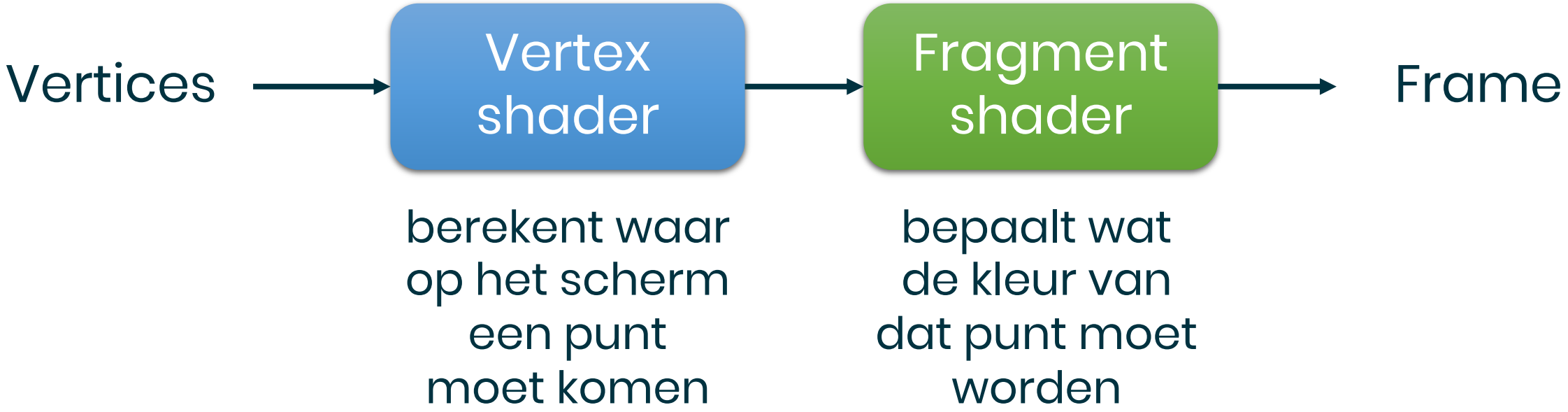
Planning

1. Fragment shaders
2. Vertex shaders



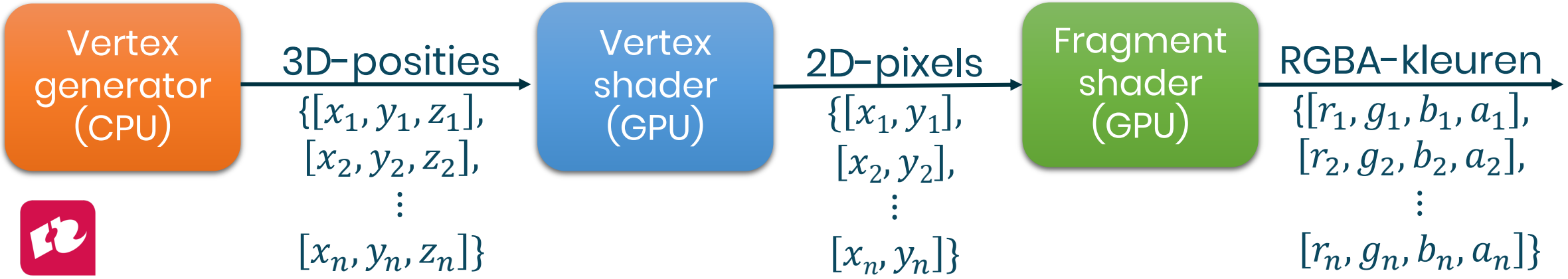
Concept: Graphics *pijplijn*

- **Input**: lijst 3D-hoekpunten (vertices)
- **Output**: gerenderd plaatje (frame)



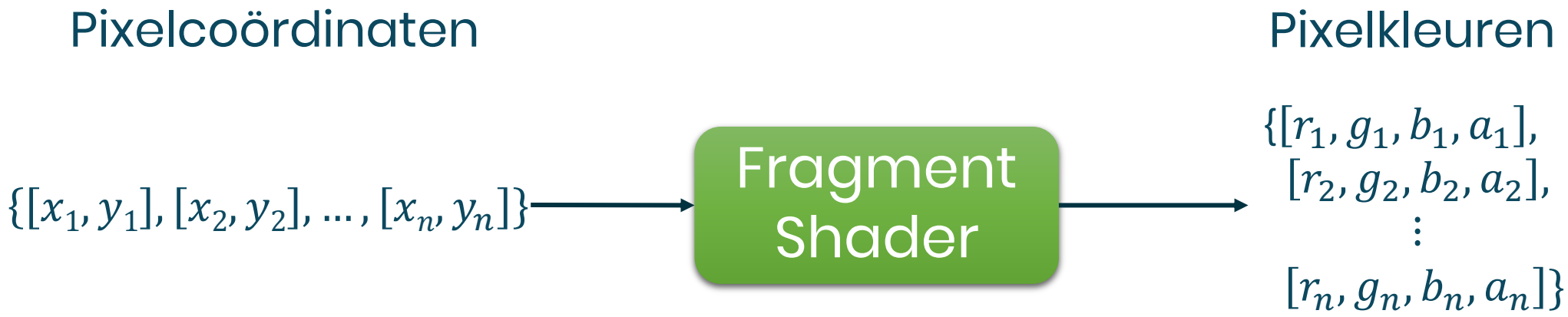
Shaders

- Programma's die draaien op de GPU in plaats van de CPU.
- Geschreven in shader-taal GLSL (lijkt op C).
- **Vertex shader**:
  - Krijgt een 3D-punt in de ruimte en geeft 2D-pixelcoördinaten terug.
  - Wordt toegepast op alle vertices van wat we willen tekenen.
- **Fragment shader**:
  - Krijgt 2D-pixelcoördinaten en geeft een RGBA-kleur terug.
  - Wordt toegepast op alle pixels van het *fragment* (punt, lijn of driehoek).



Fragment shader

- Functie die de *shade* van ieder pixel bepaalt.
- Ook wel *pixel shader* genoemd.
- Moet schrijven naar variabele `fragColor` (een 4-vector: r, g, b, alpha).
- Kan parallel draaien op meerdere cores van de GPU die elk een eigen gebied van het scherm bedienen.



Oefening: Fragment shader in ShaderToy

- Ga naar https://www.shadertoy.com/, een platform om (fragment) shaders te testen en ermee te spelen.
- Klik op “new” om een nieuwe shader te maken.
- Plak deze code (zie GitLab) en klik op het driehoekje om te runnen.

```
vec4 circle(vec2 p) {
  const float r = 100.0; // radius
  if (p.x * p.x + p.y * p.y <= r * r) {
    return vec4(1, 0, 0, 1); // red
  }
  return vec4(0, 1, 0, 1); // green
}
void mainImage(out vec4 fragColor, in vec2 fragCoord) {
  vec2 c = fragCoord - 0.5 * iResolution.xy; // shift origin to centre
  fragColor = circle(c);
}
```



Oefening: Fragment shader in ShaderToy

- Probeer in plaats van `circle()` ook `checkBoard()`:

```
vec4 checkBoard(vec2 c) {
  const float s = 50.0; // size
  vec2 p = floor(c / s);
  if (int(p.x + p.y) % 2 == 0) {
    return vec4(1, 0, 0, 1); // red
  }
  return vec4(0, 1, 0, 1); // green
}
```

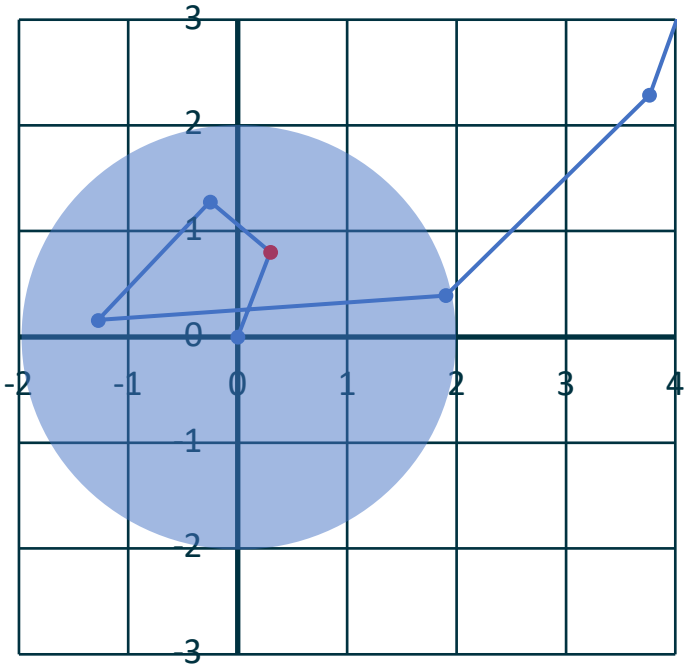
- Kun je in plaats van een cirkel ook andere vormen maken, bijv. een vierkant of een driehoek?



Demo: Mandelbrot-set

- **Mandelbrot-set**: een soort fractal (zelf-herhalend patroon).
- Een pixel valt ofwel binnen de set, ofwel er buiten.
- Voor ieder pixel apart te berekenen:

1. Begin met het pixel $\vec{c} = (c_x, c_y)$ en een punt $\vec{z} = (0, 0)$.
2. Bereken een nieuwe $\vec{z} = ({z_x}^2 - {z_y}^2 + c_x, 2 \cdot z_x \cdot z_y + c_y)$.
3. Als $|z| > 2$, dan valt punt $\vec{c}$ buiten de set ⇒ stop.
4. Ga terug naar stap 2.

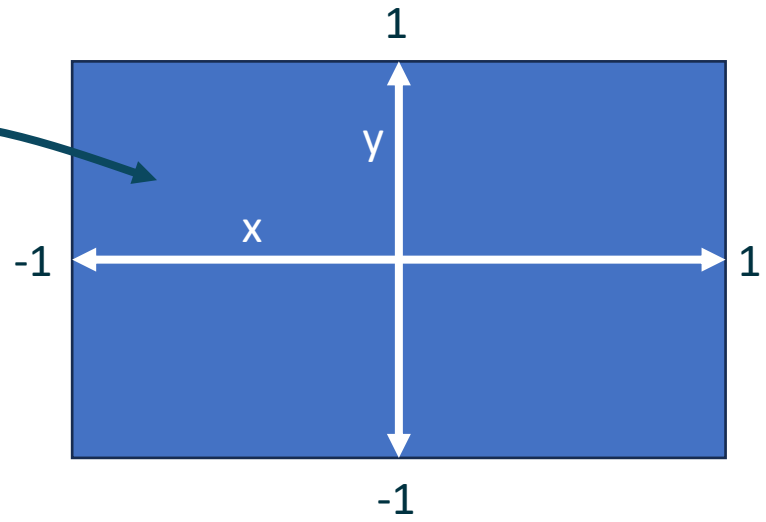


- Als je deze loop een aantal keer (~100) gedaan hebt, dan is het waarschijnlijk dat punt c binnen de set valt ⇒ stop.

Pauze (15min)

Vertex shader

- Geeft aan welke **positie** een *fragment* moet krijgen.
- Moet schrijven naar speciale variabele `gl_Position`.
- Vector van 4 componenten:
  `gl_Position = vec4(x, y, z, w);`
- `x` en `y`: positie in *clipping space*.
- `z`: voor *depth buffer* (komt later).
- `w`: hier wordt alles door gedeeld;
  gebruik `1.0` voor nu.



3D-posities

$\{[x_1, y_1, z_1],$
$[x_2, y_2, z_2],$
$\vdots$
$[x_n, y_n, z_n]\}$

Vertex shader

2D-posities

$\{[x'_1, y'_1, 0, 1],$
$[x'_2, y'_2, 0, 1],$
$\vdots$
$[x'_n, y'_n, 0, 1]\}$

Shader workflow

Een C++ programma dat gebruik maakt van shaders moet:

1. Zorgen voor een OpenGL *context* (het venster waarin gerenderd wordt).
2. Een GLSL shader-programma compileren om op de grafische kaart uitgevoerd te worden door de GPU.
3. De data (vertices) ook op de grafische kaart zetten.
4. De data koppelen aan bepaalde variabelen ("*attributes*") in het shader-programma.
5. Een tekeninstructie geven om het shader-programma te laten uitvoeren.



GLFW-library

- Zorgt voor de *context* waarin je GL-functies kunt gebruiken.
- Maakt een venster en handelt de input van muis en toetsenbord af.
- Hebben we al gebruikt in de draw-library.

```
#include <GLFW/glfw3.h>
 ⋮
glfwInit();
```

- Voor shaders hebben we een *core profile* context nodig met versie 3.3:

```
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
GLFWwindow* window = glfwCreateWindow(640, 480, "Shader example", NULL, NULL);
glfwMakeContextCurrent(window);
```



GLEW-library

- Omdat ieder GPU-model weer andere mogelijkheden (extensions) heeft, moet er een tussenlaag zijn die ervoor zorgt dat de juiste functies beschikbaar zijn.
- Dit is wat de GLEW-library (GL Extension Wrangler) doet.
- Installatie op Windows: `pacman -S mingw-w64-ucrt-x86_64-glew`
- Gebruik:

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
⋮
// after context is available:
glewInit()
```



Vertex shader

- Shader code moet beginnen met *version string*: `#version 330 core`
- Kan input krijgen: *attribute*
  - Declareer *attribute* met keyword `in`.
  - `pos` wordt later ingevuld.
- Moet schrijven naar speciale variabele **`gl_Position`**.
- Doet alle berekeningen voor transformaties en projectie (dit voorbeeld nog niet).

```
#version 330 core

in vec3 pos;

void main() {
  gl_Position = vec4(pos, 1.0);
}
```



Vertex shader compileren

- De source code van de shader is opgenomen in een *raw string* of kan als apart bestand ingelezen worden.
- *Raw string*:
  - begint met `R"(` en eindigt met `)"`
  - geen escape-codes
  - newlines worden letterlijk overgenomen

```
GLuint vs = glCreateShader(GL_VERTEX_SHADER);
const char *vsSource =
R"(#version 330 core
   in vec3 pos;
   void main() {
     gl_Position = vec4(pos, 1.0);
   })";
glShaderSource(vs, 1, &vsSource, NULL);
glCompileShader(vs);
```



Fragment shader

- Moet een output variabele van type `vec4` definiëren met keyword `out`.

```
GLuint fs = glCreateShader(GL_FRAGMENT_SHADER);
const char *fsSource =
R"(#version 330 core
   out vec4 fragColor;
   void main() {
     fragColor = vec4(0.5, 0.5, 1.0, 1.0);
   })";
glShaderSource(fs, 1, &fsSource, NULL);
glCompileShader(fs);
```

- Deze shader definieert een kleur (0.5, 0.5, 1.0; lichtblauw) en een opaciteit (1.0; ondoorzichtig) en kent die toe aan `fragColor`.
- Dit geeft dus dezelfde kleur voor alle fragments (pixels) van de figuur.



Shader program

- De twee shaders moeten gecombineerd worden tot één *shader program*:

```
GLuint program = glCreateProgram();
glAttachShader(program, vs);
glAttachShader(program, fs);
glLinkProgram(program);
glUseProgram(program);
```



Data

- De data voor de vertices moet naar een *Vertex Buffer Object* (VBO) in het geheugen van de GPU gekopieerd worden.
- Hiervoor hebben we ook een *Vertex Array Object* (VAO) nodig, waarin bijgehouden wordt hoe de data gestructureerd is.
- Je kan meerdere VAO's en VBO's hebben.
- Om aan te geven welke je wilt gebruiken, moet je ze *binden*.

```
GLfloat vertices[] = {
  -1,  0, 0,
   0,  1, 0,
   1, -1, 0
};
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);
GLuint vbo;
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glBufferData(GL_ARRAY_BUFFER,
  sizeof(vertices), vertices,
  GL_STATIC_DRAW);
```



Data koppelen

- De data in de VBO moet aan een variabele (“*attribute*”) in het shaderprogramma gekoppeld worden.
- Nodig: de *locatie* van het attribute in het geheugen:

```
GLint posLocation = glGetAttribLocation(program, "pos");
glVertexAttribPointer(posLocation, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(posLocation);
```

- 3 getallen per vertex voor dit attribute (x, y en z)
- Getaltype is `float`
- Niet normaliseren (n.v.t. bij `float`)
- Standaard *stride* (totaal aantal bytes per vertex)
- Geen offset



Tekenen

- In een loop wordt het hele plaatje telkens opnieuw getekend.
- Daarna wachten totdat een *event* optreedt (bijv. window geresized, muisklik, toetsaanslag) .

```
while (!glfwWindowShouldClose(window)) {
  glClear(GL_COLOR_BUFFER_BIT);
  glDrawArrays(GL_LINE_LOOP, 0, 3);
  glfwSwapBuffers(window);
  glfwWaitEvents();
}
```

- GL_LINE_LOOP geeft aan hoe de punten getekend worden.
- 3 is het aantal te tekenen vertices.



Opdracht: Mijn eerste shader

- Installeer de GLEW library:

Windows / MSYS2

```
pacman -S mingw-w64-ucrt-x86_64-glew
```

MacOS

```
brew install glew
```

- Compileer het voorbeeldprogramma (zie GitLab).
- Probeer als tekeninstructie ook `GL_TRIANGLES` in plaats van `GL_LINE_LOOP`.
- Pas het programma aan zodat er meerdere driehoeken getekend worden.
- Kun je ook andere vormen dan driehoeken tekenen, bijv. vierhoeken? Tip: `GL_TRIANGLE_STRIP`. Zie de OpenGL-documentatie.



Check-out

- Wat hebben we geleerd?
  - Wat een fragment shader is.
  - Wat een vertex shader is.
  - Hoe je shaders compileert en uitvoert vanuit C++.



Afsluiting

Verwerkingsopdracht:

- Maak een **Shader**-class met een constructor waaraan je de shader-code kan meegeven, en een method voor het compileren van de shader.
- Maak subclasses voor een vertex shader en een fragment shader.
- Pas het voorbeeldprogramma aan zodat het deze classes gebruikt.

Voorbereidingsopdracht:

- Zoek uit wat de verschillen en overeenkomsten zijn tussen:
  - Een Vertex Array Object (VAO)
  - Een Vertex Buffer Object (VBO)
  - Een Element Buffer Object (EBO)

Volgende keer:

- Vervolg 3D-shaders



HOGESCHOOL ROTTERDAM

Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden